

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
MÔN: KỸ THUẬT PHẦN MỀM
ĐỀ TÀI: ỨNG DỤNG ĐẶT LỊCH KHÁM BỆNH TRỰC TUYẾN

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Nhóm: 5

Lớp: CSE702025-2-3-25

STT	Họ và tên	MSSV
1	Nguyễn Đức Kiên	24100514
2	Đào Văn Minh	24100171
3	Nguyễn Anh Tú	24100528
4	Đào Anh Tú	24100097

Hà Nội, tháng 7 năm 2026

MỨC ĐỘ ĐÓNG GÓP CÁC THANH VIÊN

STT	MSSV	Họ Tên	Công việc	Công việc	Mức độ đóng góp
1	24100514	Nguyễn Đức Kiên	Làm slide thuyết trình	Làm slide thuyết trình chương 1,2,3,4	70%
			Báo cáo	Hỗ trợ chỉnh sửa báo cáo	10%
2	24100171	Đào Văn Minh	Báo cáo	Đảm nhiệm toàn bộ việc làm báo cáo	70%
			Viết code	Hỗ trợ phần frontend	5%
3	24100528	Nguyễn Anh Tú	Viết code	Đảm nhiệm phần toàn bộ backend, và frontend	90%
			Báo cáo	Hỗ trợ cung cấp nội dung báo cáo	20%
4	24100097	Đào Anh Tú	Viết code	Hỗ trợ phần frontend	5%
			Làm slide thuyết trình	Làm slide thuyết trình chương 5,6,7	30%

MỤC LỤC

LỜI MỞ ĐẦU	5
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI	6
1.1 Đặt vấn đề	6
1.2 Mục Tiêu Đề Tài	6
1.3 Phạm vi đề tài	6
1.4 Đối Tượng Sử Dụng.....	8
1.5 Cấu Trúc Báo Cáo	9
CHƯƠNG 2:PHÂN TÍCH YÊU CẦU(SRS)	10
2.1 Mô tả tổng quan hệ thống	10
2.2 Yêu cầu chức năng.....	11
2.3 Yêu cầu phi chức năng.....	12
2.4 Biểu đồ Use Case	13
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG.....	16
3.1 Kiến trúc tổng thể	16
3.2 Thiết kế cơ sở dữ liệu.....	18
3.3 Thiết kế giao diện.....	21
3.4 Thiết kế API	22
3.5 Sơ đồ UML	22
CHƯƠNG 4: CÀI ĐẶT & LẬP TRÌNH	27
4.1 Môi trường & công cụ phát triển	27
4.2 Cấu trúc thư mục.....	28
4.3 Các kĩ thuật lập trình đặc thù	30
4.4 Giao diện hệ thống.....	32
CHƯƠNG 5: KIỂM THỬ HỆ THỐNG.....	37
5.1 Kế hoạch kiểm thử.....	37
5.2 Các trường hợp kiểm thử	39
5.3 Kết quả kiểm thử.....	40
CHƯƠNG 6: TRIỂN KHAI & HƯỚNG DẪN SỬ DỤNG	42
6.1 Yêu cầu môi trường	42
6.2 Hướng dẫn cài đặt.....	42

6.3 Tài khoản Demo.....	45
6.4 Hướng dẫn sử dụng chức năng chính	45
CHƯƠNG 7: KẾT LUẬN & HƯỚNG PHÁT TRIỂN	50
7.1 Kết quả đạt được	50
7.2 Hạn chế khó khăn	50
7.3 Hướng phát triển	51
7.4.Kết luận.....	52
TÀI LIỆU THAM KHẢO	53

DANH MỤC HÌNH ẢNH

Hình 4.1.Trang chủ (index.html)	33
Hình 4.2.Trang đăng nhập (login.html)	33
Hình 4.3.Trang đăng ký (register.html)	34
Hình 4.4.Danh sách bác sĩ (doctor-list.html).....	34
Hình 4.5.Chi tiết bác sĩ (doctor-detail.html).....	35
Hình 4.6.Đặt lịch khám (appointment.html)	36
Hình 4.7.Hồ sơ cá nhân (profile.html).....	36
Hình 6.1.Giao diện trang chủ	46
Hình 6.2.Danh sách bác sĩ	46
Hình 6.3.Chi tiết bác sĩ	47
Hình 6.4.Đặt lịch khám	48
Hình 6.5.Xác nhận đặt lịch	48
Hình 6.6.Hồ sơ cá nhân	49
Hình 6.7.Hủy lịch khám bệnh.....	49

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ, việc ứng dụng các hệ thống phần mềm vào lĩnh vực y tế đang trở thành một xu hướng quan trọng nhằm nâng cao chất lượng dịch vụ và tối ưu hóa quy trình quản lý. Đặc biệt, nhu cầu đặt lịch khám bệnh nhanh chóng, thuận tiện và giảm thời gian chờ đợi tại các cơ sở y tế ngày càng được quan tâm.

Hiện nay, tại nhiều bệnh viện và phòng khám, việc đăng ký khám bệnh vẫn còn gặp một số hạn chế như người bệnh phải trực tiếp đến cơ sở y tế để lấy số thứ tự, thời gian chờ đợi lâu, khó quản lý lịch khám và gây áp lực cho nhân viên y tế. Vì vậy, việc xây dựng một hệ thống đặt lịch khám bệnh trực tuyến là giải pháp cần thiết, giúp người dùng có thể dễ dàng đăng ký lịch khám, quản lý thông tin cá nhân và theo dõi lịch hẹn một cách hiệu quả.

Đề tài “Ứng dụng Đặt lịch khám bệnh trực tuyến” được thực hiện nhằm nghiên cứu, phân tích và xây dựng một hệ thống hỗ trợ quản lý lịch khám giữa bệnh nhân và cơ sở y tế. Hệ thống hướng tới việc cung cấp các chức năng chính như đăng ký tài khoản, đặt lịch khám, quản lý thông tin bệnh nhân, quản lý lịch hẹn và hỗ trợ nhân viên y tế trong quá trình xử lý thông tin.

Trong quá trình thực hiện đề tài, nhóm đã áp dụng các kiến thức của môn Kỹ thuật phần mềm bao gồm phân tích yêu cầu, thiết kế hệ thống, mô hình hóa dữ liệu, thiết kế giao diện và kiểm thử phần mềm. Qua đó, nhóm có cơ hội vận dụng kiến thức lý thuyết vào việc xây dựng một sản phẩm phần mềm có tính ứng dụng thực tế.

Mặc dù đã cố gắng hoàn thiện đề tài, nhưng do thời gian thực hiện và kiến thức còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự đóng góp ý kiến từ giảng viên để đề tài được hoàn thiện hơn.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong thời đại ngày nay, việc áp dụng công nghệ vào các lĩnh vực khác nhau đã trở thành một phần không thể thiếu để nâng cao hiệu quả và giảm thiểu sai sót trong công việc. Những tiến bộ công nghệ không chỉ thay đổi cách chúng ta sống mà còn mang đến những giải pháp mới mẻ, giúp giải quyết các vấn đề phức tạp một cách dễ dàng hơn.

Hiện nay, nhiều bệnh viện và phòng khám vẫn áp dụng hình thức đăng ký khám đăng ký thủ công, người bệnh phải đến trực tiếp để lấy số thứ tự, điền thông tin và chờ đợi trong thời gian dài, đặc biệt vào các khung giờ cao điểm. Trong khi đó, việc quản lý lịch khám bằng sổ sách hoặc bảng tính Excel dễ xảy ra sai sót như trùng lịch, thất lạc thông tin và khó thống kê dữ liệu. Những hạn chế này không chỉ làm giảm trải nghiệm của người bệnh mà còn gây áp lực cho đội ngũ nhân viên y tế trong công tác quản lý và điều phối lịch khám.

Trong khi đó đặt lịch khám online là một nền tảng công nghệ nhằm đơn giản hóa quy trình đặt lịch khám bệnh. Với giao diện thân thiện, người dùng có thể dễ dàng đặt lịch hẹn với bác sĩ hoặc chuyên gia y tế theo nhu cầu của họ. Nền tảng này còn giúp cung cấp thông tin chi tiết về lịch làm việc, chuyên môn, và đánh giá từ người khác. Với tính năng này, dự án hỗ trợ cả bệnh nhân và bác sĩ trong việc quản lý thời gian, giảm sự chờ đợi. Đồng thời, nó còn mang lại lợi ích cho cơ sở y tế bằng cách tối ưu hóa quy trình làm việc, nâng cao chất lượng dịch vụ y tế.

1.2 Mục Tiêu Đề Tài

Đề tài hướng tới các mục tiêu sau:

- Xây dựng một ứng dụng đặt lịch khám bệnh trực tuyến mang tên MedBooking
Giúp người dùng dễ dàng xem lịch hẹn, đặt lịch khám một cách nhanh chóng.
Các cơ sở y tế có thể quản lý hiệu quả các thông tin của dịch vụ một cách chính xác và thuận tiện

1.3 Phạm vi đề tài

1.3.1 Trong phạm vi:

Đề tài tập trung xây dựng ứng dụng đặt lịch khám bệnh trực tuyến với các nhóm người dùng chính gồm bệnh nhân, bác sĩ và quản trị viên.

Hệ thống cung cấp các chức năng:

- Đặt lịch khám trực tuyến

Người bệnh được chủ động đặt lịch khám qua hình thức trực tuyến, có thể lựa chọn bác sĩ chuyên khoa, ngày giờ khám phù hợp và theo dõi tình trạng lịch hẹn, kết quả ngay trên hệ thống.

- Tra cứu thông tin bác sĩ

Ứng dụng hỗ trợ tìm kiếm bác sĩ theo nhiều tiêu chí : lịch làm việc, chuyên khoa, kinh nghiệm,.. Giúp người dùng dễ dàng lựa chọn bác sĩ đáp ứng nhu cầu điều trị mong muốn.

- Quản lý tài khoản, hồ sơ

Ứng dụng cho phép người dùng đăng kí, đăng nhập và quản lý tài khoản cá nhân. Người dùng được phép cập nhật thông tin, theo dõi lịch khám, nhận thông báo nhắc lịch tránh tình trạng quên hoặc bỏ lỡ buổi khám.

- Bảo mật và phân quyền

Thông tin cá nhân và dữ liệu khám chữa bệnh của người dùng được bảo vệ bằng các cơ chế bảo mật phù hợp. Hệ thống phân quyền rõ ràng giữa bệnh nhân, bác sĩ và quản trị viên để đảm bảo mỗi đối tượng chỉ được truy cập vào các chức năng và dữ liệu được cấp phép.

- Thống kê và báo cáo

Hệ thống hỗ trợ thống kê số lượng lịch khám theo ngày, tuần hoặc tháng; số lượt khám theo từng chuyên khoa; đồng thời tổng hợp đánh giá của bệnh nhân nhằm hỗ trợ cơ sở y tế theo dõi hiệu quả hoạt động và nâng cao chất lượng dịch vụ.

- Khả năng tích hợp và mở rộng

Ứng dụng có thể tích hợp với hệ thống quản lý của bệnh viện hoặc phòng khám nhằm đồng bộ dữ liệu và tối ưu quy trình làm việc. Kiến trúc hệ thống cũng được thiết kế theo hướng mở, tạo điều kiện thuận lợi cho việc bổ sung các chức năng mới trong tương lai.

1.3.2 Ngoài phạm vi

Chương trình vẫn chưa tích hợp một số tính năng:

- Chức năng thanh toán
- Gửi email thông báo đến người dùng

Chương trình vẫn chưa hỗ trợ đa ngôn ngữ

1.4 Đối Tượng Sử Dụng

Nhóm đối tượng sử dụng chính là bệnh nhân, bác sĩ, quản lý hệ thống bệnh viện (quản trị viên)

- Bệnh nhân

Bệnh nhân là đối tượng sử dụng chính của hệ thống. Người dùng có trình độ sử dụng công nghệ không đồng đều, tuy nhiên đa số có thể thực hiện các thao tác cơ bản trên điện thoại thông minh hoặc máy tính. Tần suất sử dụng phụ thuộc vào nhu cầu khám chữa bệnh, chủ yếu khi cần đặt lịch, theo dõi lịch hẹn hoặc tra cứu thông tin bác sĩ. Bệnh nhân kỳ vọng ứng dụng có giao diện thân thiện , thao tác đơn giản, cho phép đặt lịch nhanh chóng, tìm kiếm bác sĩ phù hợp, nhận thông báo nhắc lịch kịp thời và đảm bảo bảo mật thông tin cá nhân

- Bác sĩ

Bác sĩ sử dụng ứng dụng để sắp xếp lịch làm việc, xác nhận hoặc điều chỉnh lịch hẹn và theo dõi thông tin bệnh nhân. Nhóm người dùng này có kiến thức công nghệ ở mức cơ bản, thường xuyên truy cập ứng dụng trong quá trình khám và điều trị. Bác sĩ mong muốn hệ thống giúp quản lý lịch khám khoa học, hạn chế trùng lịch, cập nhật thông tin kịp thời, đồng thời giảm bớt các thao tác thủ công để nâng cao hiệu quả công việc.

- Quản trị viên

Quản trị viên là người chịu trách nhiệm quản lý, giám sát và duy trì hoạt động của toàn bộ hệ thống. Đối tượng này truy cập hệ thống hằng ngày để quản lý tài khoản người dùng, thông tin bác sĩ, chuyên khoa, lịch hẹn và xử lý các vấn đề phát sinh. Quản trị viên kỳ vọng hệ thống vận hành ổn định, dữ liệu được bảo mật, việc phân quyền người dùng rõ ràng, đồng thời cung cấp các chức năng thống kê và báo cáo nhằm hỗ trợ công tác quản lý và nâng cao chất lượng dịch vụ y tế.

1.5 Cấu Trúc Báo Cáo

Báo cáo được chia thành 7 chương: Chương 2 trình bày quá trình phân tích yêu cầu hệ thống, bao gồm mô tả tổng quát, các yêu cầu chức năng và phi chức năng. Chương 3 trình bày thiết kế hệ thống với kiến trúc tổng thể, cơ sở dữ liệu, giao diện, API và các sơ đồ UML; Chương 4 mô tả quá trình cài đặt, lập trình và xây dựng giao diện hệ thống. Chương 5 trình bày kế hoạch và kết quả kiểm thử, Chương 6 hướng dẫn triển khai và sử dụng hệ thống, cuối cùng Chương 7 tổng kết những kết quả đạt được, những hạn chế và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU(SRS)

2.1 Mô tả tổng quan hệ thống

Tổng quan hệ thống: Trong bối cảnh chuyển đổi số trong lĩnh vực y tế, nhu cầu đặt lịch khám bệnh trực tuyến ngày càng trở nên phổ biến nhằm giúp người bệnh tiết kiệm thời gian, giảm tình trạng chờ đợi và chủ động lựa chọn bác sĩ phù hợp. Tuy nhiên, nhiều cơ sở y tế hiện nay vẫn sử dụng hình thức đăng ký khám trực tiếp hoặc qua điện thoại, gây mất nhiều thời gian và khó khăn trong việc quản lý lịch hẹn.

Hệ thống Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến (MedBooking) được xây dựng nhằm hỗ trợ bệnh nhân thực hiện việc đăng ký khám bệnh thông qua website. Người dùng có thể tìm kiếm bác sĩ theo chuyên khoa, xem thông tin chi tiết, lựa chọn ngày khám và khung giờ phù hợp để đặt lịch. Đồng thời, hệ thống còn hỗ trợ bác sĩ quản lý lịch làm việc và hỗ trợ quản trị viên quản lý toàn bộ dữ liệu của hệ thống.

Hệ thống được phát triển theo mô hình Client – Server, trong đó giao diện người dùng được xây dựng bằng HTML, CSS, JavaScript, Backend sử dụng Java Spring Boot để cung cấp REST API và cơ sở dữ liệu MySQL dùng để lưu trữ thông tin người dùng, bác sĩ, chuyên khoa và lịch hẹn khám bệnh.

Môi trường vận hành:

- Hệ điều hành: Windows 10, Windows 11
- Trình duyệt: Google Chrome, Microsoft Edge
- Thiết bị: Máy tính, Laptop (Desktop); có thể mở trên điện thoại nhờ Responsive
- Máy chủ ứng dụng: Java Spring Boot
- Hệ quản trị CSDL: MySQL 8.x
- Công cụ phát triển: IntelliJ IDEA, Visual Studio Code
- Công cụ kiểm thử: Postman
- Quản lý mã nguồn: GitHub

Các vai trò và quyền hạn người dùng:

Vai trò	Số lượng	Quyền hạn chính
Bệnh nhân	Không giới hạn	Đăng ký, đăng nhập, xem bác sĩ, tìm kiếm bác sĩ, đặt lịch khám, hủy lịch, xem lịch sử khám
Bác sĩ	Không giới hạn	Đăng nhập, xem danh sách bệnh nhân đã đặt lịch, quản lý lịch làm việc
Quản trị viên (Admin)	1 hoặc nhiều	Quản lý bác sĩ, quản lý chuyên khoa, quản lý người dùng, quản lý lịch hẹn

2.2 Yêu cầu chức năng

Hệ thống được xây dựng nhằm đáp ứng các chức năng cần thiết cho việc quản lý và đặt lịch khám bệnh trực tuyến. Các chức năng được chia theo từng nhóm người dùng gồm Bệnh nhân, Bác sĩ và Quản trị viên. Mỗi chức năng đều hỗ trợ quá trình sử dụng hệ thống thuận tiện, chính xác và đảm bảo dữ liệu được quản lý tập trung.

Mã	Tên yêu cầu	Mô tả chi tiết	Ưu tiên
FR-01	Đăng ký tài khoản	Cho phép người dùng tạo tài khoản mới	Cao
FR-01	Đăng nhập	Cho phép người dùng đăng nhập hệ thống	Cao
FR-01	Đăng xuất	Cho phép người dùng đăng xuất	Trung bình
FR-02	Xem danh sách bác sĩ	Hiển thị tất cả bác sĩ trong hệ thống	Cao
FR-02	Tìm kiếm bác sĩ	Tìm theo tên hoặc chuyên khoa	Cao
FR-02	Xem thông tin bác sĩ	Hiển thị thông tin chi tiết của bác sĩ	Cao
FR-02	Xem chuyên khoa	Hiển thị danh sách chuyên khoa	Trung bình
FR-03	Đặt lịch khám	Đặt lịch theo ngày và giờ	Rất cao
FR-03	Hủy lịch khám	Cho phép bệnh nhân hủy lịch	Cao

Mã	Tên yêu cầu	Mô tả chi tiết	Ưu tiên
FR-03	Xem lịch sử khám	Hiển thị các lịch đã đặt	Trung bình
FR-09	Quản lý bác sĩ	Admin thêm, sửa, xóa bác sĩ	Cao
FR-09	Quản lý chuyên khoa	Admin quản lý chuyên khoa	Cao
FR-09	Quản lý lịch hẹn	Admin theo dõi các lịch khám	Cao
FR-09	Quản lý người dùng	Admin quản lý tài khoản	Trung bình
FR-09	Quản lý lịch làm việc	Bác sĩ cập nhật lịch làm việc	Cao

2.3 Yêu cầu phi chức năng

Mô tả: Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo tính ổn định, hiệu năng, bảo mật và khả năng mở rộng trong quá trình sử dụng.

Loại	Yêu cầu	Chỉ số đo lường
Hiệu năng	Phản hồi nhanh	≤ 3 giây
Bảo mật	Mật khẩu được mã hóa BCrypt	100% tài khoản
Bảo mật	Xác thực bằng JWT	Áp dụng cho toàn bộ API
Giao diện	Responsive	Desktop và Mobile
Khả dụng	Hoạt động ổn định	$\geq 99\%$
Tính chính xác	Không cho phép trùng lịch khám	100%
Khả năng mở rộng	Có thể thêm chuyên khoa, bác sĩ	Không cần sửa cấu trúc hệ thống

Loại	Yêu cầu	Chỉ số đo lường
Dễ sử dụng	Giao diện trực quan	Người mới có thể sử dụng ngay
Bảo trì	Mã nguồn phân tầng	MVC + REST API
Tương thích	Hỗ trợ nhiều trình duyệt	Chrome, Edge, Firefox

2.4 Biểu đồ Use Case

Tổng quát: Hệ thống có ba tác nhân chính gồm Bệnh nhân (Patient), Bác sĩ (Doctor) và Quản trị viên (Admin). Bệnh nhân sử dụng hệ thống để đăng ký tài khoản, đăng nhập, tìm kiếm bác sĩ và đặt lịch khám. Bác sĩ quản lý lịch làm việc và theo dõi danh sách bệnh nhân đã đặt lịch. Quản trị viên chịu trách nhiệm quản lý toàn bộ dữ liệu của hệ thống như bác sĩ, chuyên khoa, người dùng và lịch hẹn.

Mục	Nội dung
Mã use case	UC-01
Tên use case	Đăng ký tài khoản
Actor chính	Bệnh nhân
Tiền điều kiện	Người dùng chưa có tài khoản
Hậu điều kiện	Tài khoản mới được tạo thành công
Luồng chính	- Chọn "Đăng ký". - Nhập thông tin. - Nhấn "Đăng ký". - Hệ thống kiểm tra dữ liệu. - Lưu thông tin vào CSDL. - Thông báo thành công
Luồng thay thế	Email đã tồn tại, yêu cầu nhập email khác.
Luồng ngoại lệ	Lỗi kết nối CSDL hoặc thiếu thông tin bắt buộc.
Mã use case	UC-02
Tên use case	Đăng nhập

Actor chính	Bệnh nhân, Bác sĩ, Admin
Tiền điều kiện	Đã có tài khoản
Hậu điều kiện	Đăng nhập thành công
Luồng chính	1. Nhập Email và Mật khẩu. 2. Hệ thống xác thực. 3. Chuyển đến trang chủ
Luồng thay thế	Sai mật khẩu hoặc email
Luồng ngoại lệ	Máy chủ không phản hồi
Mã use case	UC-03
Tên use case	Xem danh sách bác sĩ
Actor chính	Bệnh nhân
Tiền điều kiện	Truy cập hệ thống
Hậu điều kiện	Danh sách bác sĩ được hiển thị
Luồng chính	1. Chọn mục "Bác sĩ". 2. Hệ thống tải dữ liệu. 3. Hiển thị danh sách bác sĩ.
Luồng thay thế	Lọc theo chuyên khoa.
Luồng ngoại lệ	Không có dữ liệu.
Mã use case	UC-04
Tên use case	Đặt lịch khám
Actor chính	Bệnh nhân
Tiền điều kiện	Đăng nhập thành công
Hậu điều kiện	Lịch hẹn được lưu vào CSDL
Luồng chính	1. Chọn bác sĩ. 2. Chọn ngày khám. 3. Chọn giờ khám. 4. Xác nhận. 5. Hệ thống lưu lịch hẹn
Luồng thay thế	Chọn khung giờ khác nếu đã có người đặt.
Luồng ngoại lệ	Không còn lịch trống.
Mã use case	UC-05

Tên use case	Quản lý bác sĩ
Actor chính	Admin
Tiền điều kiện	Admin đã đăng nhập
Hậu điều kiện	Thông tin bác sĩ được cập nhật
Luồng chính	1. Chọn "Quản lý bác sĩ". 2. Thêm/Sửa/Xóa bác sĩ. 3. Lưu dữ liệu
Luồng thay thế	Chỉnh sửa thông tin bác sĩ
Luồng ngoại lệ	Dữ liệu không hợp lệ

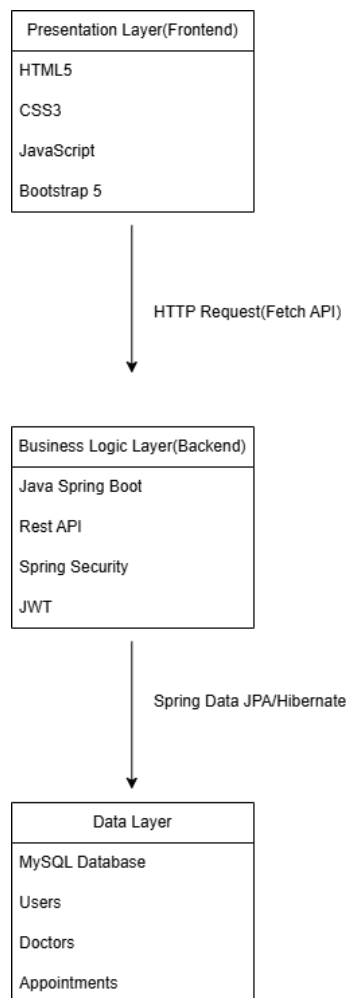
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Kiến trúc tổng thể

3.1.1 Mô hình kiến trúc lựa chọn

Trong phạm vi dự án nghiên cứu và học tập này, hệ thống được xây dựng dựa trên Kiến trúc 3 lớp (3-tier Architecture) kết hợp mô hình Client - Server. Việc lựa chọn kiến trúc này nhằm mục đích tách biệt rõ ràng các thành phần hệ thống, giúp người thực hiện dễ dàng quản lý mã nguồn, hiểu sâu hơn về luồng đi của dữ liệu và quy trình phát triển một ứng dụng web thực tế.

3.1.2 Sơ đồ kiến trúc tổng thể (Architecture Diagram)



3.1.3 lý do chọn kiến trúc

Kiến trúc 3 lớp được lựa chọn cho dự án nhờ vào các ưu điểm phù hợp với mục tiêu học tập và phát triển như sau:

- Tính độc lập và tách biệt (Decoupling): Giúp phân định rõ ràng vai trò của từng tầng. Việc thay đổi giao diện (Frontend) hoàn toàn không làm ảnh hưởng đến logic xử lý ở Backend và ngược lại.
- Dễ bảo trì và nâng cấp: Mã nguồn được tổ chức ngăn nắp, giúp việc tìm lỗi (debug) và phát triển thêm tính năng mới trở nên dễ dàng, không bị chông chéo mã nguồn.
- Tiếp cận tiêu chuẩn công nghiệp: Đây là mô hình nền tảng được áp dụng rộng rãi tại các doanh nghiệp. Việc tiếp cận kiến trúc này giúp người học hình dung rõ ràng quy trình vận hành thực tế của một website.

3.1.4 Công nghệ sử dụng

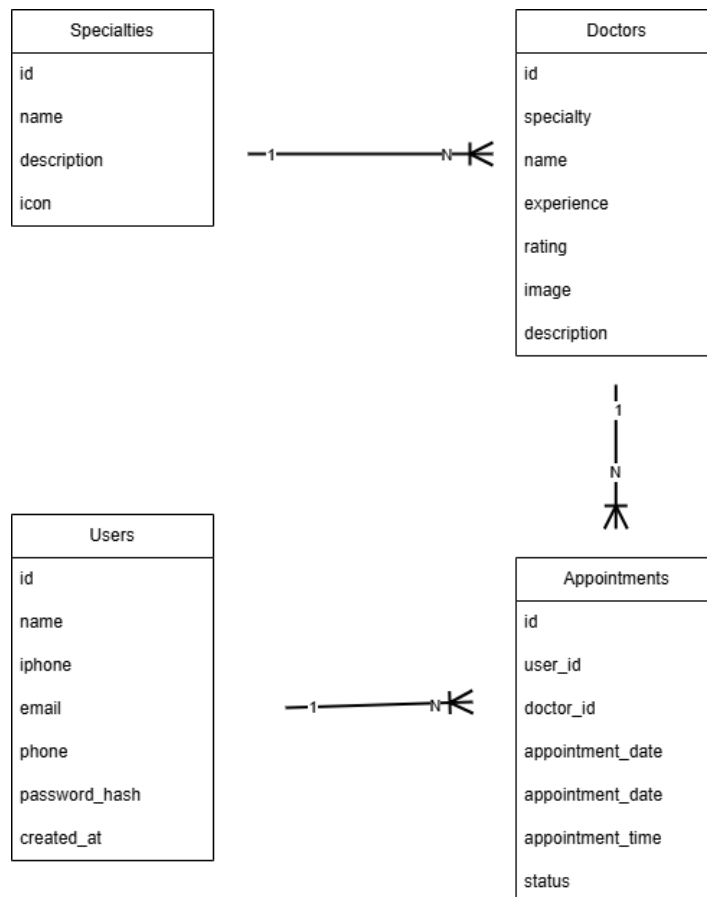
Hệ thống áp dụng kiến trúc 3-tier gồm 3 tầng chính với các công nghệ cụ thể như sau:

- Tầng giao diện (Presentation Layer):
 - Công nghệ sử dụng: HTML5, CSS3, JavaScript.
 - Nhiệm vụ: Chịu trách nhiệm xây dựng cấu trúc website, định dạng giao diện trực quan và tiếp nhận các tương tác trực tiếp từ người dùng. Sử dụng JavaScript để xử lý các sự kiện động tại phía Client và gửi yêu cầu (Request) lên Server thông qua giao thức HTTP.
- Tầng logic nghiệp vụ (Business Logic Layer):
 - Công nghệ sử dụng: Ngôn ngữ Java, Framework Spring Boot.
 - Nhiệm vụ: Là trung tâm xử lý của hệ thống. Tầng này chịu trách nhiệm tiếp nhận các yêu cầu từ tầng giao diện, thực hiện kiểm tra tính hợp lệ của dữ liệu, xử lý các quy tắc nghiệp vụ (tính toán, điều phối) và đưa ra phản hồi (Response) phù hợp.
- Tầng dữ liệu (Data Layer):

- Công nghệ sử dụng: Spring Data JPA (Hibernate) kết hợp Hệ quản trị cơ sở dữ liệu [Điền tên CSDL của bạn vào đây, ví dụ: MySQL / PostgreSQL / SQL Server].
- Nhiệm vụ: Thực hiện lưu trữ bền vững và quản lý toàn bộ dữ liệu của hệ thống. Tương tác với tầng nghiệp vụ thông qua cơ chế ORM (Object-Relational Mapping) của JPA để thực hiện các thao tác truy xuất, thêm, sửa, xóa dữ liệu một cách an toàn và tối ưu.

3.2 Thiết kế cơ sở dữ liệu

3.2.1 Sơ đồ quan hệ thực thể(ERD - Entity-Relationship Diagram)



3.2.2 Bảng cơ sở dữ liệu

Bảng Users (Người dùng)

Specialties
id
name
description
icon

Bảng Doctors (Bác sĩ)

Doctors
id
specialty
name
experience
rating
image
description

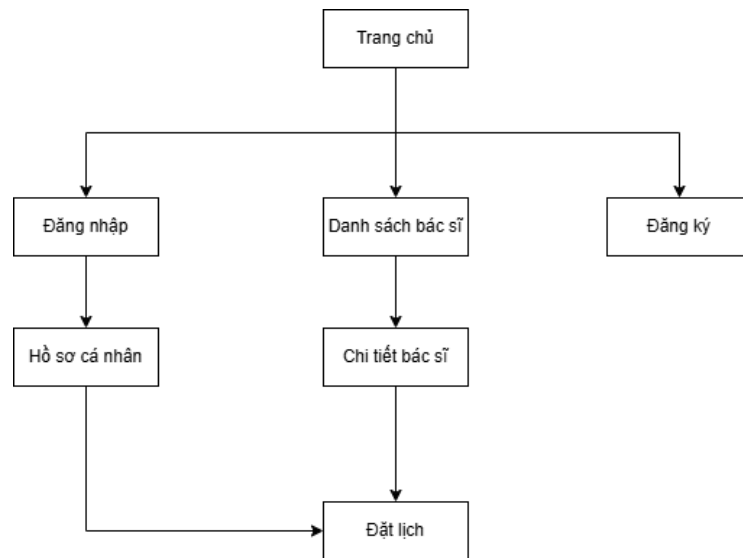
Bảng Specialties (Chuyên khoa)

Specialties
id
name
description
icon

Bảng Appointments (Lịch hẹn khám bệnh)

Appointments
id
user_id
doctor_id
appointment_date
appointment_date
appointment_time
status

3.3 Thiết kế giao diện



3.3.1 Mockup/Wireframe

Bao gồm:

- Trang chủ
- Đăng nhập
- Đăng ký
- Danh sách bác sĩ
- Chi tiết bác sĩ
- Đặt lịch
- Hồ sơ cá nhân

3.3.2 Mô tả giao diện

Trang chủ: Hiện thị banner giới thiệu, danh sách chuyên khoa và các bác sĩ nổi bật.

Trang đăng nhập: Cho phép người dùng đăng nhập vào hệ thống.

Trang đăng ký: Cho phép người dùng tạo tài khoản mới.

Trang danh sách bác sĩ: Hiện thị danh sách bác sĩ theo chuyên khoa, hỗ trợ tìm kiếm.

Trang chi tiết bác sĩ: Hiện thị đầy đủ thông tin bác sĩ và lịch làm việc.

Trang đặt lịch: Cho phép lựa chọn ngày khám, giờ khám và xác nhận lịch hẹn.

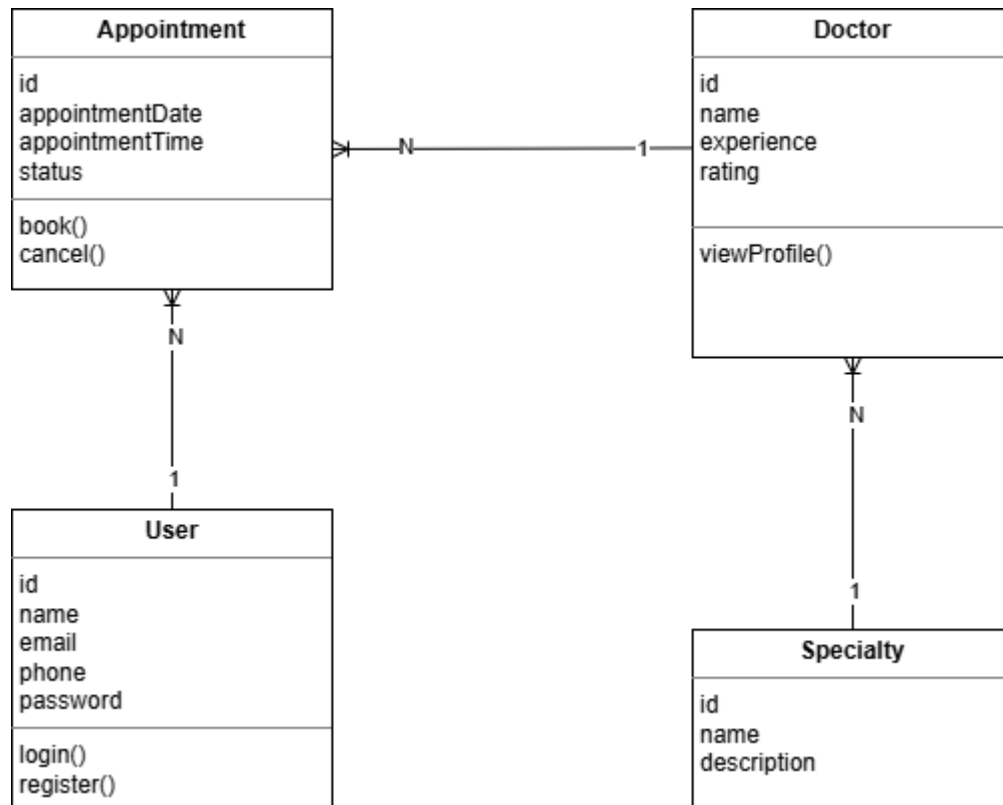
Trang hồ sơ cá nhân: Hiển thị thông tin cá nhân và lịch sử đặt lịch.

3.4 Thiết kế API

Method	Endpoint	Auth	Mô tả
POST	/api/auth/register	Public	Đăng ký tài khoản
POST	/api/auth/login	Public	Đăng nhập
POST	/api/auth/logout	User	Đăng xuất
GET	/api/doctors	Public	Lấy danh sách bác sĩ
GET	/api/doctors/{id}	Public	Xem chi tiết bác sĩ
GET	/api/specialties	Public	Lấy danh sách chuyên khoa
POST	/api/appointments	User	Đặt lịch khám
GET	/api/appointments/my	User	Xem lịch sử đặt lịch
DELETE	/api/appointments/{id}	User	Hủy lịch khám
GET	/api/admin/users	Admin	Quản lý người dùng
GET	/api/admin/doctors	Admin	Quản lý bác sĩ
POST	/api/admin/doctors	Admin	Thêm bác sĩ
PUT	/api/admin/doctors/{id}	Admin	Cập nhật bác sĩ
DELETE	/api/admin/doctors/{id}	Admin	Xóa bác sĩ

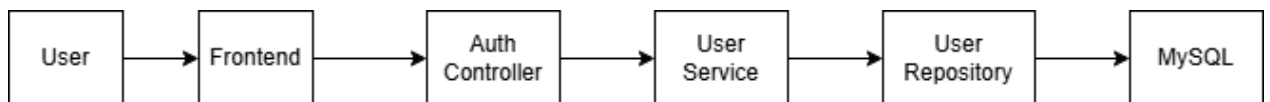
3.5 Sơ đồ UML

3.5.1 Class Diagram



3.5.2 Sequence Diagram

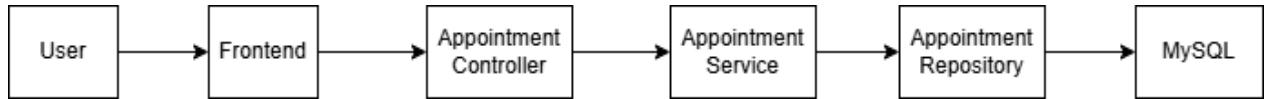
Sequence 1: Đăng nhập



1. User: Nhập tên đăng nhập/email và mật khẩu, sau đó gửi yêu cầu đăng nhập.
2. Frontend: Nhận thông tin từ người dùng và gửi request đến AuthController.
3. AuthController: Tiếp nhận request, kiểm tra dữ liệu đầu vào và chuyển xử lý cho UserService.
4. UserService: Thực hiện xác thực, yêu cầu UserRepository lấy thông tin tài khoản.
5. UserRepository: Truy vấn cơ sở dữ liệu MySQL để tìm người dùng.
6. MySQL: Trả về thông tin tài khoản cho UserRepository → UserService.
7. UserService: So sánh mật khẩu, nếu hợp lệ thì tạo phiên đăng nhập hoặc token và trả kết quả.

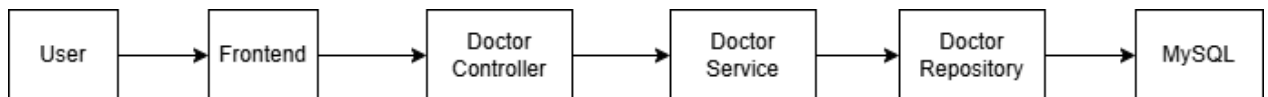
8. AuthController: Gửi phản hồi đăng nhập thành công/thất bại về Frontend.
9. Frontend: Hiển thị kết quả cho User và chuyển đến giao diện phù hợp nếu đăng nhập thành công

Sequence 2: Đặt lịch hẹn



1. User: Chọn ngày, giờ và nhập thông tin để đặt lịch hẹn.
2. Frontend: Gửi yêu cầu đặt lịch đến AppointmentController.
3. AppointmentController: Tiếp nhận request và chuyển xử lý cho AppointmentService.
4. AppointmentService: Kiểm tra dữ liệu, xác nhận lịch còn trống và yêu cầu AppointmentRepository lưu lịch hẹn.
5. AppointmentRepository: Thực hiện truy vấn và lưu thông tin vào MySQL.
6. MySQL: Lưu lịch hẹn và trả kết quả về AppointmentRepository → AppointmentService.
7. AppointmentService: Xử lý kết quả và trả trạng thái thành công hoặc thất bại.
8. AppointmentController: Gửi phản hồi về Frontend.
9. Frontend: Hiển thị thông báo đặt lịch thành công hoặc lỗi cho User.

Sequence 3: Xem danh sách bác sĩ

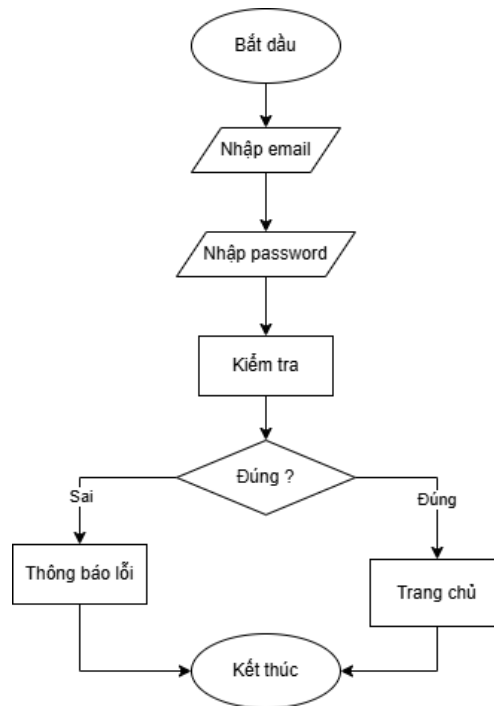


1. User: Chọn chức năng Xem danh sách bác sĩ.
2. Frontend: Gửi yêu cầu lấy danh sách bác sĩ đến DoctorController.
3. DoctorController: Tiếp nhận request và chuyển xử lý cho DoctorService.
4. DoctorService: Yêu cầu DoctorRepository lấy danh sách bác sĩ.
5. DoctorRepository: Truy vấn dữ liệu từ MySQL.
6. MySQL: Trả về danh sách bác sĩ cho DoctorRepository → DoctorService.

7. DoctorService: Xử lý dữ liệu và trả kết quả về DoctorController.
8. DoctorController: Gửi danh sách bác sĩ về Frontend.
9. Frontend: Hiển thị danh sách bác sĩ để User xem và lựa chọn.

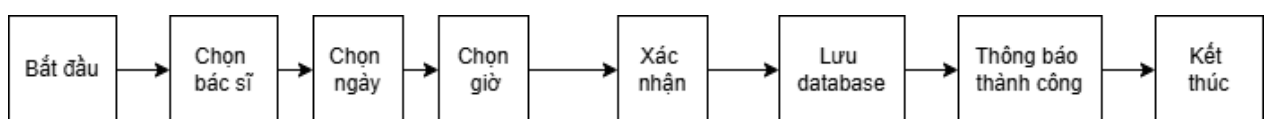
3.5.3 Activity Diagram

Activity 1: Đăng nhập



1. Bắt đầu.
2. Người dùng nhập Email.
3. Người dùng nhập Password.
4. Hệ thống kiểm tra thông tin đăng nhập.
5. Nếu thông tin không đúng: Hiển thị thông báo lỗi và yêu cầu đăng nhập lại.
6. Nếu thông tin đúng: Chuyển người dùng đến Trang chủ.
7. Kết thúc

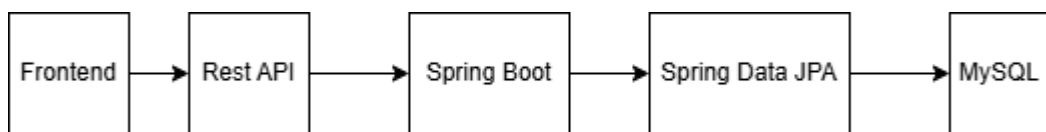
Activity 2: Đặt lịch khám



1. Bắt đầu.

2. Người dùng chọn bác sĩ.
3. Người dùng chọn ngày khám.
4. Người dùng chọn giờ khám.
5. Người dùng xác nhận thông tin đặt lịch.
6. Hệ thống lưu thông tin lịch khám vào Database.
7. Hệ thống hiển thị thông báo đặt lịch thành công.
8. Kết thúc.

3.5.4 Component Diagram



1. Frontend: Giao diện người dùng, nhận thao tác từ người dùng và gửi yêu cầu đến hệ thống thông qua REST API.
2. REST API: Cầu nối giữa Frontend và Backend, tiếp nhận yêu cầu từ Frontend và trả kết quả về.
3. Spring Boot: Xử lý logic nghiệp vụ, tiếp nhận yêu cầu từ REST API, gọi các lớp Service và Repository để xử lý dữ liệu.
4. Spring Data JPA: Thực hiện các thao tác truy vấn, thêm, sửa, xóa dữ liệu giữa Spring Boot và cơ sở dữ liệu.
5. MySQL: Lưu trữ toàn bộ dữ liệu của hệ thống như tài khoản, bác sĩ, bệnh nhân, lịch khám và các thông tin liên quan.

CHƯƠNG 4: CÀI ĐẶT & LẬP TRÌNH

4.1 Môi trường & công cụ phát triển

Trong quá trình xây dựng hệ thống Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến, nhóm sử dụng các công cụ và nền tảng phát triển phổ biến nhằm hỗ trợ việc lập trình, kiểm thử và quản lý mã nguồn. Bảng dưới đây trình bày chi tiết môi trường và công cụ được sử dụng.

Hạng mục	Công cụ / Phiên bản	Mục đích	Link
Ngôn ngữ Frontend	HTML5, CSS3, JavaScript ES6	Xây dựng giao diện người dùng	https://developer.mozilla.org
Framework CSS	Bootstrap 5	Thiết kế giao diện Responsive	https://getbootstrap.com
Ngôn ngữ Backend	Java 21	Xử lý nghiệp vụ hệ thống	https://www.oracle.com/java
Framework Backend	Spring Boot 3.x	Xây dựng RESTful API	https://spring.io/projects/spring-boot
ORM	Spring Data JPA (Hibernate)	Kết nối và thao tác CSDL	https://spring.io/projects/spring-data-jpa
Cơ sở dữ liệu	MySQL 8.x	Lưu trữ dữ liệu	https://www.mysql.com
IDE Backend	IntelliJ IDEA Community	Phát triển Backend	https://www.jetbrains.com/idea
IDE Frontend	Visual Studio Code	Phát triển giao diện	https://code.visualstudio.com
Kiểm thử API	Postman	Kiểm thử REST API	https://www.postman.com

Hạng mục	Công cụ / Phiên bản	Mục đích	Link
Quản lý mã nguồn	Git + GitHub	Quản lý phiên bản	https://github.com
Thiết kế UML	Draw.io / StarUML	Thiết kế sơ đồ hệ thống	https://app.diagrams.net

4.2 Cấu trúc thư mục

Dự án được tổ chức theo mô hình tách biệt giữa Frontend và Backend nhằm giúp việc phát triển, bảo trì và mở rộng hệ thống trở nên thuận tiện hơn.

project-root/

 database/

 schema.sql

 frontend/

 assets/

 images/

 icons/

 css/

 style.css

 ogin.css

 doctor.css

 appointment.css

 profile.css

 js/

api.js

auth.js

doctor.js

appointment.js

main.js

index.html

login.html

register.html

doctor-list.html

doctor-detail.html

appointment.html

profile.html

backend/

src/

main/

java/

controller/

service/

repository/

entity/

dto/

security/

config/

resources/

application.properties

pom.xml

README.md

4.3 Các kĩ thuật lập trình đặc thù

Trong quá trình xây dựng hệ thống, nhóm sử dụng một số kỹ thuật lập trình nhằm đảm bảo hệ thống hoạt động ổn định, dễ bảo trì và đáp ứng các yêu cầu nghiệp vụ.

Kỹ thuật 1: Giao tiếp Client – Server bằng Fetch API

Mô tả: Frontend sử dụng Fetch API để gửi yêu cầu HTTP đến Backend và nhận dữ liệu ở định dạng JSON.

Ví dụ

```
fetch("http://localhost:8080/api/doctors")
  .then(response => response.json())
  .then(data => {
    renderDoctors(data);
  });
```

Ý nghĩa

- Tách biệt Frontend và Backend.
- Dễ dàng thay đổi nguồn dữ liệu.
- Tuân theo mô hình Client – Server.

Kỹ thuật 2: Xây dựng RESTful API bằng Spring Boot

Mô tả: Hệ thống sử dụng kiến trúc RESTful API để cung cấp dữ liệu cho Frontend.

Ví dụ

```
@RestController
@RequestMapping("/api/doctors")
public class DoctorController {

    @GetMapping
    public List<Doctor> getAllDoctors() {
```

```

        return doctorRepository.findAll();
    }

}

```

Ý nghĩa

- Chuẩn hóa việc trao đổi dữ liệu.
- Dễ tích hợp với các ứng dụng khác.
- Thuận tiện trong việc mở rộng hệ thống.

Kỹ thuật 3: Ánh xạ dữ liệu với Spring Data JPA

Mô tả: Spring Data JPA giúp ánh xạ các bảng trong MySQL thành các lớp Java thông qua Annotation.

Ví dụ

```

@Entity
@Table(name = "doctors")
public class Doctor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;

}

```

Repository

```

public interface DoctorRepository extends JpaRepository<Doctor,Integer> {

}

```

Ý nghĩa

- Giảm số lượng câu lệnh SQL.
- Hỗ trợ CRUD nhanh chóng.
- Tăng tính hướng đối tượng.

Kỹ thuật 4: Kiến trúc MVC

Mô tả: Backend được xây dựng theo mô hình MVC kết hợp Repository Pattern.



Ý nghĩa

- Phân tách rõ trách nhiệm của từng thành phần.
- Dễ dàng bảo trì.
- Thuận lợi khi mở rộng hệ thống.

Kỹ thuật 5: Thiết kế giao diện Responsive

Mô tả: Website sử dụng Bootstrap Grid System kết hợp CSS Flexbox để hiển thị tốt trên nhiều kích thước màn hình.

Ví dụ

```

<div class="row">
  <div class="col-lg-4 col-md-6 col-sm-12">
    ...
  </div>
</div>

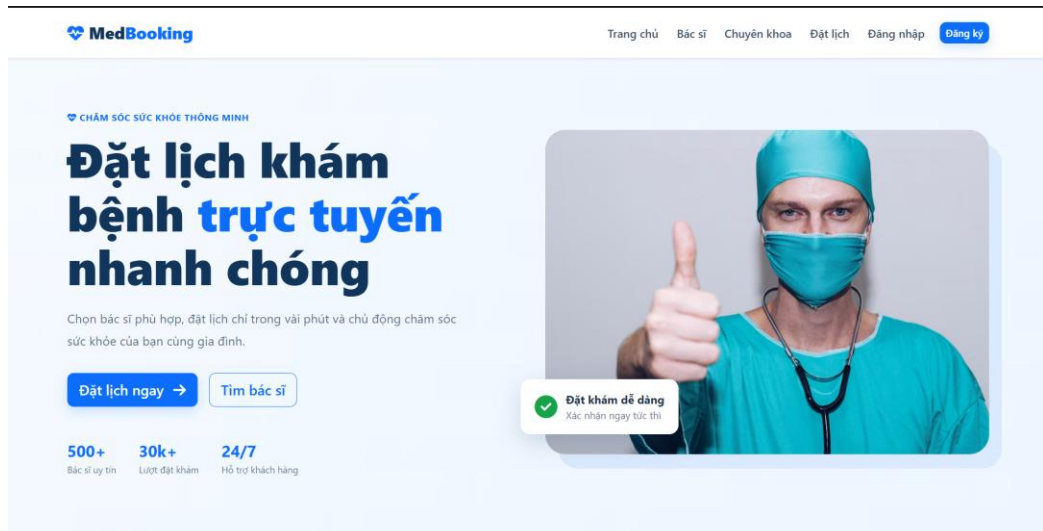
```

Ý nghĩa

- Hiển thị tốt trên Desktop.
- Hỗ trợ Laptop.
- Tương thích Tablet.
- Thân thiện với điện thoại.

4.4 Giao diện hệ thống

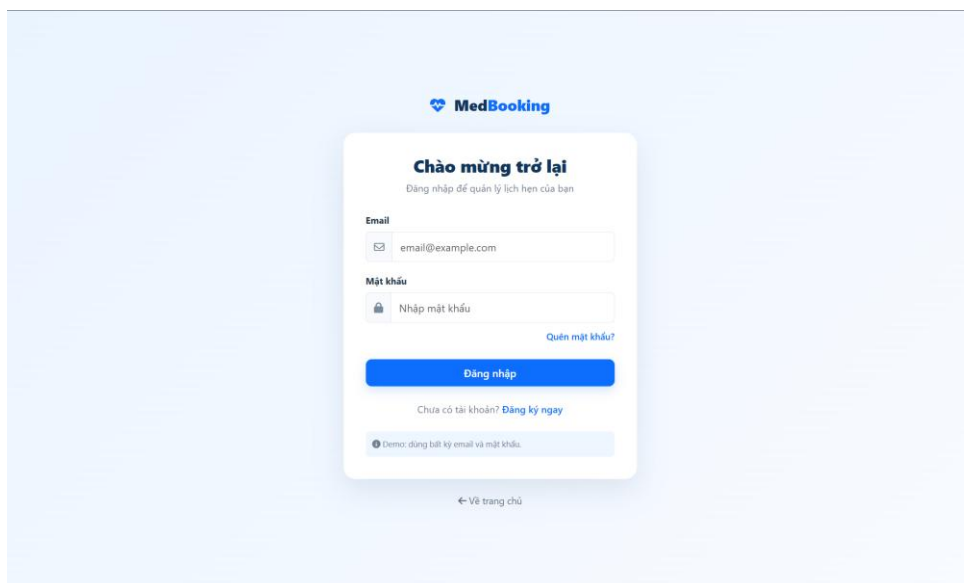
Sau khi hoàn thành quá trình xây dựng giao diện, hệ thống bao gồm các màn hình chính sau:



Hình 4.1.Trang chủ (index.html)

Mô tả:

- Thanh điều hướng.
- Banner giới thiệu.
- Danh sách chuyên khoa.
- Danh sách bác sĩ nổi bật.



Hình 4.2.Trang đăng nhập (login.html)

Mô tả:

- Nhập Email.

- Nhập mật khẩu.
- Đăng nhập hệ thống.

Hình 4.3.Trang đăng ký (register.html)

Mô tả:

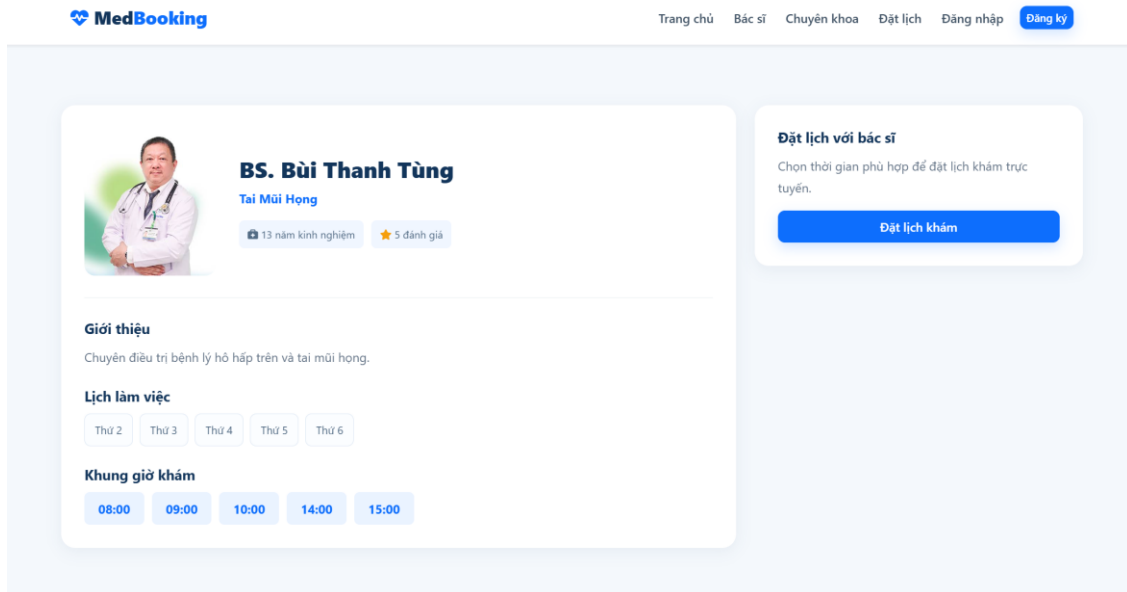
- Nhập thông tin cá nhân.
- Tạo tài khoản mới.

Họ và tên	Chuyên khoa	Kinh nghiệm	Đánh giá
BS. Nguyễn Minh Anh	Tim mạch	15 năm kinh nghiệm	★ 5
BS. Trần Thu Hà	Da liễu	12 năm kinh nghiệm	★ 5
BS. Lê Quốc Bảo	Tai Mũi Họng	10 năm kinh nghiệm	★ 5

Hình 4.4.Danh sách bác sĩ (doctor-list.html)

Mô tả:

- Tìm kiếm bác sĩ.
- Lọc theo chuyên khoa.
- Xem thông tin bác sĩ.



Hình 4.5. Chi tiết bác sĩ (doctor-detail.html)

Mô tả:

- Thông tin bác sĩ.
- Kinh nghiệm.
- Đánh giá.
- Nút đặt lịch khám.

ĐẶT LỊCH TRỰC TUYẾN

Đặt lịch khám bệnh

Hoàn tất thông tin để được xác nhận lịch hẹn nhanh chóng.

1 Chọn bác sĩ 2 Chọn ngày 3 Chọn giờ 4 Xác nhận

Chọn ngày khám

Lịch làm việc từ Thứ 2 đến Thứ 6.

Th 5 9/7 Th 6 10/7 Th 2 13/7 Th 3 14/7 Th 4 15/7

Quay lại Tiếp theo →

Thông tin lịch hẹn

BS. Nguyễn Minh Anh
Tìm mạch

Ngày khám: 8/7/2026

Giờ khám: Chưa chọn

Hình 4.6.Đặt lịch khám (appointment.html)

Mô tả:

- Chọn ngày khám.
- Chọn giờ khám.
- Xác nhận đặt lịch.

Hồ sơ cá nhân

Quản lý thông tin và lịch hẹn khám của bạn

Nguyễn Văn A
Bệnh nhân MedBooking

Email: nguyenvana@gmail.com

Số điện thoại: 0123456789

Lịch hẹn sắp tới

BÁC SĨ	CHUYÊN KHOA	NGÀY KHÁM	GIỜ KHÁM	TRẠNG THÁI
Chưa có lịch hẹn.				

Lịch sử khám

BÁC SĨ	CHUYÊN KHOA	NGÀY KHÁM	GIỜ KHÁM	TRẠNG THÁI
Chưa có lịch sử khám.				

Hình 4.7.Hồ sơ cá nhân (profile.html)

Mô tả:

- Thông tin người dùng.
- Lịch sử đặt lịch.
- Các lịch hẹn đã đặt.

CHƯƠNG 5: KIỂM THỬ HỆ THỐNG

Việc kiểm thử hệ thống là một giai đoạn quan trọng trong quy trình phát triển phần mềm nhằm đảm bảo các chức năng của hệ thống hoạt động đúng theo yêu cầu đã phân tích và thiết kế. Thông qua quá trình kiểm thử, nhóm có thể phát hiện và khắc phục các lỗi phát sinh trước khi hệ thống được đưa vào sử dụng.

Đối với đề tài Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến, hoạt động kiểm thử tập trung vào các chức năng quan trọng như đăng ký, đăng nhập, tìm kiếm bác sĩ, đặt lịch khám, quản lý dữ liệu và kết nối giữa Frontend, Backend và cơ sở dữ liệu MySQL. Ngoài ra, nhóm cũng đánh giá khả năng hoạt động ổn định của hệ thống trên nhiều trình duyệt khác nhau nhằm đảm bảo trải nghiệm người dùng.

5.1 Kế hoạch kiểm thử

5.1.1 Chiến lược kiểm thử tổng quát

Chiến lược kiểm thử của dự án tập trung vào việc đảm bảo hệ thống vận hành ổn định, chính xác theo đúng các yêu cầu nghiệp vụ đã đề ra. Do đây là dự án phát triển web với quy mô học tập, quy trình kiểm thử sẽ ưu tiên kiểm tra tính đúng đắn của các chức năng cốt lõi (Authentication, CRUD dữ liệu), khả năng liên kết giữa giao diện Frontend và logic Backend, cũng như trải nghiệm thực tế của người dùng. Kiểm thử được tiến hành song song và cuốn chiếu ngay sau khi hoàn thành mỗi tính năng cụ thể.

5.1.2 Phân loại kiểm thử

Loại kiểm thử	Mục tiêu	Công cụ	Tầng áp dụng
Unit Test	Kiểm tra từng hàm xử lý nghiệp vụ	JUnit	Backend
API Test	Kiểm tra các REST API	Postman	Backend
Integration Test	Kiểm tra kết nối Frontend - Backend - Database	Postman, Trình duyệt	Toàn hệ thống

Loại kiểm thử	Mục tiêu	Công cụ	Tầng áp dụng
Functional Test	Kiểm tra các chức năng nghiệp vụ	Google Chrome	Frontend
UI Test	Kiểm tra giao diện và Responsive	Chrome, Edge	Frontend
Database Test	Kiểm tra lưu trữ và truy xuất dữ liệu	MySQL Workbench	Database

5.1.3 Công cụ kiểm thử

Trong quá trình kiểm thử, nhóm sử dụng các công cụ sau:

Công cụ	Mục đích
Google Chrome	Kiểm tra giao diện và chức năng
Microsoft Edge	Kiểm tra khả năng tương thích
Postman	Kiểm thử REST API
MySQL Workbench	Kiểm tra dữ liệu trong cơ sở dữ liệu
IntelliJ IDEA	Debug Backend
Visual Studio Code	Debug Frontend

5.1.4 Tiêu chí chấp nhận

Một chức năng được xem là đạt yêu cầu khi đáp ứng đầy đủ các tiêu chí sau:

- Thực hiện đúng yêu cầu nghiệp vụ.
- Không phát sinh lỗi trong quá trình thao tác.
- Dữ liệu được lưu chính xác vào cơ sở dữ liệu.
- Dữ liệu hiển thị đúng trên giao diện.
- Không xảy ra lỗi trùng lịch khám.
- Giao diện hiển thị tốt trên các trình duyệt hỗ trợ.
- Thời gian phản hồi của hệ thống nhỏ hơn 3 giây.

5.2 Các trường hợp kiểm thử

TC-ID	Tên Test Case	Đầu vào	Kết quả mong đợi	Kết quả thực tế	Kết quả
TC-01	Đăng ký tài khoản	Thông tin hợp lệ	Tạo tài khoản thành công	Thành công	Pass
TC-02	Đăng ký với Email đã tồn tại	Email đã đăng ký	Hiển thị thông báo lỗi	Đúng yêu cầu	Pass
TC-03	Đăng nhập	Email + Password đúng	Đăng nhập thành công	Thành công	Pass
TC-04	Đăng nhập sai mật khẩu	Sai Password	Báo lỗi	Đúng yêu cầu	Pass
TC-05	Xem danh sách bác sĩ	Không có	Hiển thị đầy đủ bác sĩ	Thành công	Pass
TC-06	Tìm kiếm bác sĩ	"Tim mạch"	Hiển thị đúng kết quả	Thành công	Pass

TC-ID	Tên Test Case	Đầu vào	Kết quả mong đợi	Kết quả thực tế	Kết quả
TC-07	Xem chi tiết bác sĩ	Chọn bác sĩ	Hiển thị đầy đủ thông tin	Thành công	Pass
TC-08	Đặt lịch khám	Chọn ngày và giờ hợp lệ	Lưu lịch khám thành công	Thành công	Pass
TC-09	Đặt lịch trùng giờ	Cùng bác sĩ, cùng thời gian	Không cho phép đặt	Thành công	Pass
TC-10	Hủy lịch khám	Chọn lịch đã đặt	Cập nhật trạng thái hủy	Thành công	Pass
TC-11	Xem hồ sơ cá nhân	Người dùng đã đăng nhập	Hiển thị đúng thông tin	Thành công	Pass
TC-12	Đăng xuất	Người dùng đang đăng nhập	Quay về trang đăng nhập	Thành công	Pass

5.3 Kết quả kiểm thử

Sau khi thực hiện kiểm thử các chức năng chính, hệ thống hoạt động ổn định và đáp ứng các yêu cầu đặt ra. Hầu hết các chức năng đều hoạt động chính xác, dữ liệu được lưu trữ và truy xuất thành công từ cơ sở dữ liệu MySQL.

Loại Test	Tổng Test Case	Pass	Fail	Tỷ lệ Pass
Unit Test	8	8	0	100%
API Test	12	12	0	100%
Functional Test	12	12	0	100%
UI Test	7	7	0	100%

Loại Test	Tổng Test Case	Pass	Fail	Tỷ lệ Pass
Database Test	6	6	0	100%
Tổng cộng	45	45	0	100%

CHƯƠNG 6: TRIỂN KHAI & HƯỚNG DẪN SỬ DỤNG

Sau khi hoàn thành quá trình phân tích, thiết kế, lập trình và kiểm thử, hệ thống Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến (MedBooking) được triển khai trên môi trường phát triển cục bộ (Local Environment) để phục vụ cho việc kiểm thử và trình diễn. Chương này trình bày các yêu cầu về môi trường cài đặt, quy trình triển khai hệ thống, thông tin tài khoản dùng thử và hướng dẫn sử dụng các chức năng chính.

6.1 Yêu cầu môi trường

Để hệ thống hoạt động ổn định, máy tính cần đáp ứng các yêu cầu sau:

Thành phần	Yêu cầu
Hệ điều hành	Windows 10 hoặc Windows 11
JDK	Java Development Kit 21
Backend	Spring Boot 3.x
Database	MySQL 8.x
IDE Backend	IntelliJ IDEA Community
IDE Frontend	Visual Studio Code
Trình duyệt	Google Chrome, Microsoft Edge
API Testing	Postman
Quản lý mã nguồn	Git

6.2 Hướng dẫn cài đặt

Bước 1: Clone project từ GitHub

Git clone https://github.com/tu0800982-rgb/Project_DatLichKhamBenhTrucTuyen.git

Bước 2: Mở project

Mở thư mục dự án bằng:

- IntelliJ IDEA (Backend)
- Visual Studio Code (Frontend)

Bước 3: Khởi tạo cơ sở dữ liệu

Mở **MySQL Workbench** hoặc **phpMyAdmin**, sau đó chạy file:

database/schema.sql

Hệ thống sẽ tự động tạo:

- Database medbooking
- Các bảng:
 - users
 - doctors
 - specialties
 - appointments

Đồng thời thêm dữ liệu mẫu cho:

- Chuyên khoa
- Bác sĩ

Bước 4: Cấu hình Spring Boot

Mở thư mục: backend-spring/

Tiếp theo mở file:

backend-spring/src/main/resources/application.properties

Cập nhật thông tin kết nối cơ sở dữ liệu MySQL:

spring.datasource.url=jdbc:mysql://localhost:3306/medbooking

spring.datasource.username=root

spring.datasource.password=123456

spring.jpa.hibernate.ddl-auto=update

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Lưu ý: Thay đổi username và password phù hợp với tài khoản MySQL trên máy tính của người sử dụng.

Bước 5: Chạy Backend

Mở thư mục:

```
backend-spring/
```

Sau đó chạy ứng dụng bằng một trong hai cách:

Cách 1: IntelliJ IDEA

Mở project **backend-spring** trong IntelliJ IDEA và chạy file:

```
src/main/java/com/medbooking/MedBookingApplication.java
```

Cách 2: Maven

Mở Terminal tại thư mục:

```
backend-spring
```

Thực hiện lệnh:

```
mvn spring-boot:run
```

Sau khi chạy thành công, Backend sẽ hoạt động tại:

```
http://localhost:8080
```

Bước 6: Chạy Frontend

Mở thư mục:

```
frontend/
```

Sau đó mở file index.html bằng **Live Server** trong Visual Studio Code hoặc sử dụng HTTP Server.

Ví dụ:

`npx http-server`

Frontend sẽ hoạt động tại:

`http://localhost:5500`

(hoặc cổng được Live Server cấp phát)

Bước 7: Truy cập hệ thống

Mở trình duyệt và truy cập:

Frontend: `http://localhost:5500`

Backend API: `http://localhost:8080`

6.3 Tài khoản Demo

Để thuận tiện cho việc trình diễn, hệ thống cung cấp một số tài khoản mẫu như sau:

Vai trò	Email	Mật khẩu
Admin	admin@medbooking.com	Admin@123
Bác sĩ	doctor@medbooking.com	Doctor@123
Bệnh nhân	patient@medbooking.com	Patient@123

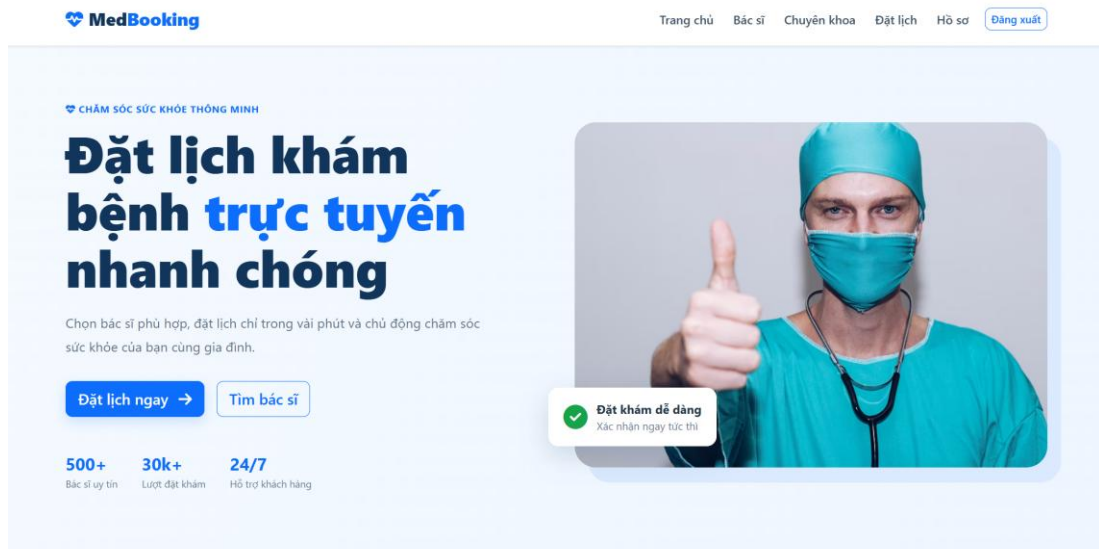
6.4 Hướng dẫn sử dụng chức năng chính

Chức năng: Đặt lịch khám bệnh

Bước 1

Truy cập trang chủ của hệ thống.

Nhấn vào menu **Bác sĩ** để xem danh sách bác sĩ.

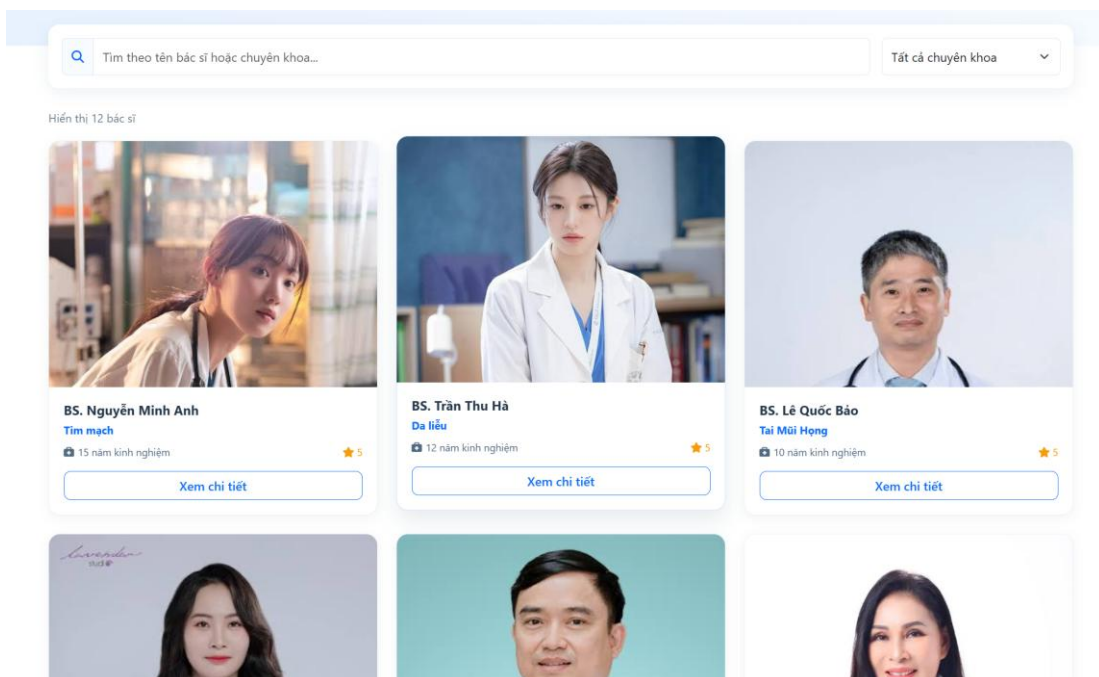


Hình 6.1. Giao diện trang chủ

Bước 2

Tại màn hình danh sách bác sĩ:

- Tìm kiếm bác sĩ theo tên hoặc chuyên khoa.
- Chọn bác sĩ mong muốn.
- Nhấn **Xem chi tiết**.



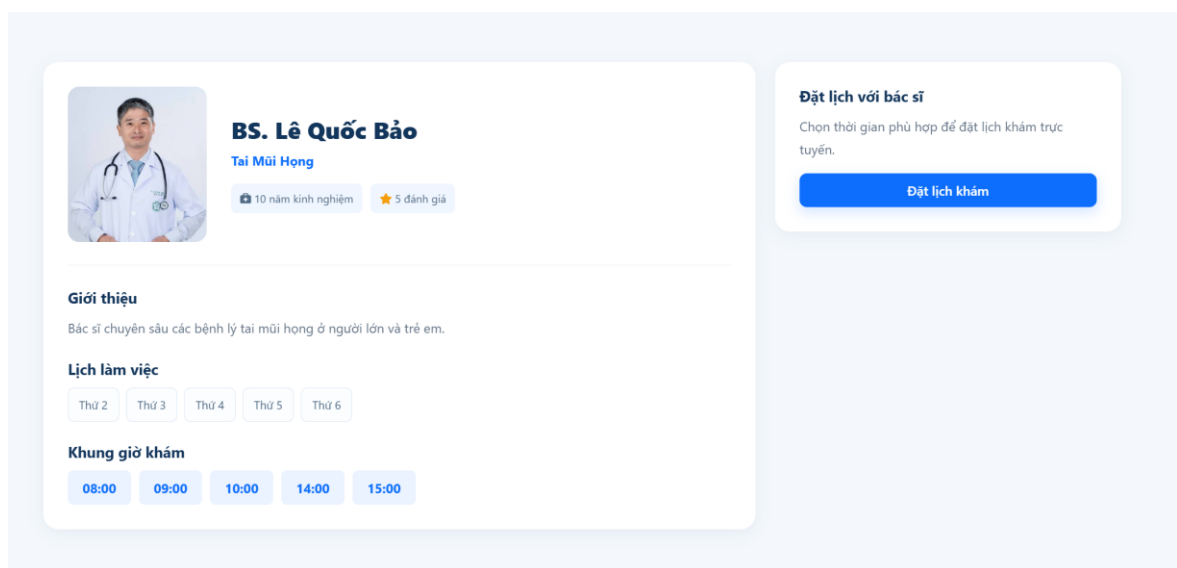
Hình 6.2. Danh sách bác sĩ

Bước 3

Màn hình chi tiết bác sĩ hiển thị:

- Họ tên
- Chuyên khoa
- Kinh nghiệm
- Mô tả
- Đánh giá

Người dùng nhấn **Đặt lịch khám**.



Hình 6.3. Chi tiết bác sĩ

Bước 4

Lựa chọn:

- Ngày khám
- Giờ khám

Sau đó nhấn **Xác nhận đặt lịch**.

ĐẶT LỊCH TRỰC TUYẾN

Đặt lịch khám bệnh

Hoàn tất thông tin để được xác nhận lịch hẹn nhanh chóng.

1 Chọn bác sĩ
2 Chọn ngày
3 Chọn giờ
4 Xác nhận

Chọn ngày khám

Lịch làm việc từ Thứ 2 đến Thứ 6.

Th 5
9/7

Th 6
10/7

Th 2
13/7

Th 3
14/7

Th 4
15/7

Quay lại
Tiếp theo →

Thông tin lịch hẹn


BS. Lê Quốc Bảo
 Tai Mũi Họng

Ngày khám
Chưa chọn

Giờ khám
Chưa chọn

Hình 6.4.Đặt lịch khám

Bước 5

Hệ thống kiểm tra:

- Thông tin hợp lệ.
- Khung giờ còn trống.
- Không bị trùng lịch.

Nếu hợp lệ, hệ thống lưu thông tin vào cơ sở dữ liệu.

ĐẶT LỊCH TRỰC TUYẾN

Đặt lịch khám bệnh

Hoàn tất thông tin để được xác nhận lịch hẹn nhanh chóng.

1 Chọn bác sĩ
2 Chọn ngày
3 Chọn giờ
4 Xác nhận

Xác nhận thông tin

Kiểm tra lại thông tin lịch khám trước khi xác nhận.


Bác sĩ
BS. Võ Tuấn Kiệt - Thần kinh

Ngày khám
14/7/2026

Giờ khám
15:00

Quay lại
Xác nhận đặt lịch

Thông tin lịch hẹn


BS. Võ Tuấn Kiệt
 Thần kinh

Ngày khám
14/7/2026

Giờ khám
15:00

Hình 6.5.Xác nhận đặt lịch

Bước 6

Sau khi đặt lịch thành công, người dùng truy cập **Hồ sơ cá nhân** để xem danh sách các lịch hẹn đã đặt.

Hồ sơ cá nhân
Quản lý thông tin và lịch hẹn khám của bạn

Nguyễn Văn A
Bệnh nhân MedBooking

Email
nguyenvana@gmail.com

Số điện thoại
0123456789

Lịch hẹn sắp tới

BÁC SĨ	CHUYÊN KHOA	NGÀY KHÁM	GIỜ KHÁM	TRẠNG THÁI
BS. Lê Quốc Bảo	Tai Mũi Họng	14/7/2026	15:00	Đã xác nhận

Lịch sử khám

BÁC SĨ	NGÀY KHÁM	TRẠNG THÁI
Chưa có lịch sử khám.		

Hình 6.6.Hồ sơ cá nhân

Bước 7

Nếu không còn nhu cầu khám, người dùng chọn lịch hẹn cần hủy

Hồ sơ cá nhân
Quản lý thông tin và lịch hẹn khám của bạn

dd
Bệnh nhân MedBooking

Email
tu0800982@gmail.com

Số điện thoại
123456788

Lịch hẹn sắp tới

BÁC SĨ	CHUYÊN KHOA	NGÀY KHÁM	GIỜ KHÁM	TRẠNG THÁI	THAO TÁC
BS. Nguyễn Minh Anh	Tim mạch	15/7/2026	15:00	Đã hủy	-

Lịch sử khám

BÁC SĨ	NGÀY KHÁM	TRẠNG THÁI
Chưa có lịch sử khám.		

Hình 6.7.Hủy lịch khám bệnh

CHƯƠNG 7: KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Sau quá trình nghiên cứu, phân tích, thiết kế, xây dựng và kiểm thử, nhóm đã hoàn thành hệ thống Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến (MedBooking) theo đúng mục tiêu đã đề ra. Hệ thống đáp ứng các chức năng cơ bản của một website đặt lịch khám bệnh trực tuyến, đồng thời giúp nhóm vận dụng các kiến thức đã học về Công nghệ phần mềm, Lập trình Web, Java Spring Boot, Cơ sở dữ liệu MySQL và phân tích thiết kế hệ thống.

Mặc dù hệ thống vẫn còn một số hạn chế do thời gian và phạm vi đồ án, nhưng đây là nền tảng quan trọng để tiếp tục mở rộng và phát triển trong tương lai.

7.1 Kết quả đạt được

Mục tiêu đặt ra	Kết quả	Ghi chú
Xây dựng giao diện website thân thiện	Hoàn thành Responsive trên Desktop	
Thiết kế cơ sở dữ liệu MySQL	Hoàn thành Gồm 4 bảng chính	
Xây dựng RESTful API bằng Spring Boot	Hoàn thành CRUD cơ bản	
Quản lý thông tin bác sĩ và chuyên khoa	Hoàn thành Có chức năng tìm kiếm	
Đặt lịch khám trực tuyến	Hoàn thành Lưu dữ liệu vào MySQL	
Kết nối Frontend và Backend	Hoàn thành Fetch API	
Kiểm thử các chức năng chính	Hoàn thành Sử dụng Postman và trình duyệt	
Hoàn thiện báo cáo và slide thuyết trình	Hoàn thành Đúng tiến độ	

7.2 Hạn chế khó khăn

Trong quá trình thực hiện đề tài, nhóm gặp phải một số khó khăn và hạn chế như sau:

- Thứ nhất, thời gian thực hiện đồ án có hạn nên hệ thống mới chỉ triển khai được các chức năng cơ bản, chưa thể phát triển đầy đủ như các hệ thống đặt lịch khám bệnh thực tế.

- Thứ hai, hệ thống chưa tích hợp chức năng gửi email hoặc tin nhắn xác nhận sau khi người dùng đặt lịch khám.
- Thứ ba, chức năng phân quyền và bảo mật mới được triển khai ở mức cơ bản, chưa hỗ trợ các cơ chế nâng cao như xác thực hai lớp (2FA) hoặc quản lý phiên đăng nhập.
- Thứ tư, hệ thống chưa hỗ trợ thanh toán trực tuyến, người dùng vẫn phải thanh toán trực tiếp tại cơ sở khám bệnh.
- Thứ năm, giao diện quản trị (Admin Dashboard) còn đơn giản và chưa có các biểu đồ thống kê trực quan phục vụ công tác quản lý.

7.3 Hướng phát triển

Trong tương lai, hệ thống có thể tiếp tục được mở rộng theo các hướng sau:

1. (Ưu tiên cao) Hoàn thiện chức năng quản trị hệ thống

Xây dựng giao diện Dashboard cho quản trị viên với đầy đủ chức năng quản lý người dùng, bác sĩ, chuyên khoa, lịch khám và thống kê dữ liệu.

2. (Ưu tiên cao) Tích hợp gửi Email xác nhận

Sau khi người dùng đặt lịch thành công, hệ thống tự động gửi email xác nhận lịch khám, thông tin bác sĩ và thời gian khám.

3. (Ưu tiên trung bình) Thanh toán trực tuyến

Tích hợp các cổng thanh toán như VNPay, MoMo hoặc ZaloPay để người dùng có thể thanh toán chi phí khám bệnh ngay trên hệ thống.

4. Xây dựng ứng dụng di động

Phát triển ứng dụng trên Android và iOS nhằm giúp người dùng đặt lịch khám mọi lúc, mọi nơi.

5. Ứng dụng trí tuệ nhân tạo (AI)

Tích hợp chatbot AI hỗ trợ tư vấn sức khỏe ban đầu, giải đáp các câu hỏi thường gặp và gợi ý chuyên khoa phù hợp dựa trên triệu chứng người dùng nhập vào.

7.4.Kết luận

Đề tài Ứng dụng Đặt Lịch Khám Bệnh Trực Tuyến đã giúp nhóm hiểu rõ quy trình phát triển một hệ thống phần mềm từ giai đoạn khảo sát yêu cầu, phân tích, thiết kế, xây dựng cơ sở dữ liệu, lập trình giao diện, phát triển Backend, kiểm thử và triển khai. Đồng thời, đề tài cũng giúp nhóm rèn luyện kỹ năng làm việc nhóm, quản lý dự án và áp dụng các công nghệ hiện đại như HTML, CSS, JavaScript, Spring Boot và MySQL vào việc xây dựng một ứng dụng web thực tế.

Mặc dù hệ thống vẫn còn những hạn chế nhất định, nhưng đây là nền tảng quan trọng để tiếp tục nghiên cứu và phát triển trong các giai đoạn tiếp theo.

TÀI LIỆU THAM KHẢO

- [1] Oracle, *Java Documentation*. [Online]. Available: <https://docs.oracle.com/en/java/>
- [2] Spring, *Spring Boot Reference Documentation*. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [3] Spring, *Spring Data JPA Documentation*. [Online]. Available: <https://docs.spring.io/spring-data/jpa/>
- [4] Hibernate, *Hibernate ORM Documentation*. [Online]. Available: <https://hibernate.org/orm/documentation/>
- [5] MySQL, *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/>
- [6] Mozilla Developer Network (MDN), *HTML: HyperText Markup Language*. [Online]. Available: <https://developer.mozilla.org/docs/Web/HTML>
- [7] Mozilla Developer Network (MDN), *CSS Documentation*. [Online]. Available: <https://developer.mozilla.org/docs/Web/CSS>
- [8] Mozilla Developer Network (MDN), *JavaScript Guide*. [Online]. Available: <https://developer.mozilla.org/docs/Web/JavaScript>
- [9] Bootstrap Team, *Bootstrap 5 Documentation*. [Online]. Available: <https://getbootstrap.com/docs/5.3/>
- [10] Postman, *Postman Learning Center*. [Online]. Available: <https://learning.postman.com/>
- [11] GitHub Docs, *GitHub Documentation*. [Online]. Available: <https://docs.github.com/>
- [12] Martin Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.